TRACKORA

Selected Technology Stack & Development Roadmap

Trackora is planned as a connected field-sales management system with a mobile application for salespeople, a web dashboard for administrators and managers, and a shared backend.

Mobile application	Flutter + Dart
Admin web application	Next.js + TypeScript
Backend API	NestJS + TypeScript
Database and ORM	PostgreSQL + Prisma
Supporting services	Firebase Cloud Messaging, SQLite/Drift, private object storage, maps and background jobs

Project direction

Use one organized repository (a monorepo) for the mobile app, admin website and backend. Both user-facing applications connect to the same backend API. Important business rules, permissions and database operations stay on the server.

Product goal: build a real connected application, not a screen-only prototype. Each phase should include runnable code, setup instructions and tests before the next phase.

1. Recommended Repository Structure

```
trackora/
■■■ apps/
■   ■■■ mobile/                # Flutter application
■   ■   ■■■ lib/
■   ■   ■   ■■■ core/
■   ■   ■   ■■■ features/
■   ■   ■   ■   ■■■ auth/
■   ■   ■   ■   ■■■ attendance/
■   ■   ■   ■   ■■■ visits/
■   ■   ■   ■   ■■■ customers/
■   ■   ■   ■   ■■■ targets/
■   ■   ■   ■   ■■■ profile/
■   ■   ■   ■■■ main.dart
■   ■   ■■■ test/
■   ■■■ admin-web/             # Next.js dashboard
■   ■   ■■■ app/
■   ■   ■■■ components/
■   ■   ■■■ features/
■   ■   ■■■ lib/
■   ■■■ api/                   # NestJS backend
■       ■■■ src/
■       ■   ■■■ auth/
■       ■   ■■■ users/
■       ■   ■■■ teams/
■       ■   ■■■ customers/
■       ■   ■■■ attendance/
■       ■   ■■■ visits/
■       ■   ■■■ approvals/
■       ■   ■■■ targets/
■       ■   ■■■ leaderboard/
■       ■   ■■■ notifications/
■       ■■■ prisma/
■           ■■■ schema.prisma
■■■ docs/
■   ■■■ architecture.md
■   ■■■ api-specification.md
■■■ README.md
```

Repository notes

- Keep mobile, web and API code in separate application folders.
- Organize each backend feature as a module with its controller, service, validation and tests.
- Use Prisma schema and migrations to track database structure changes.
- Document system architecture and API contracts in the docs folder.

2. Selected Technology Stack

Layer	Technology	Primary purpose
Mobile	Flutter + Dart	Android/iOS app, GPS, forms and offline workflows
Admin web	Next.js + TypeScript	Dashboard, employee management, approvals and reports
Backend	NestJS + TypeScript	REST APIs, business rules, authentication and permissions
Database	PostgreSQL	Persistent relational data and reporting
ORM / migrations	Prisma	Typed database access and schema migrations
Push notifications	Firebase Cloud Messaging	Push notifications to mobile devices
Offline storage	SQLite / Drift	Local drafts and a synchronization queue
Files	Private object storage	Visit photos and audio attachments
Location and maps	Device location plugin + maps provider	Location capture and map display
Background work	Task scheduler / job queue	Scheduled reminders, retries and report jobs

3. Development Roadmap

Phase 1 — Foundation

- Set up all three applications and the repository.
- Configure environment variables, PostgreSQL and Prisma migrations.
- Implement authentication and role permissions.

Phase 2 — Mobile field operations

- Implement attendance and GPS capture.
- Add customer management, visit creation and attachments.
- Build offline storage and synchronization.

Phase 3 — Admin operations

- Build employee and team management.
- Add visit review, approval and rejection workflows.
- Provide historical records and reports.

Phase 4 — Performance engine

- Add targets and approved-visit calculations.
- Implement leaderboard rules.
- Add push notifications and scheduled reminders.

Phase 5 — Production testing

- Test permissions, offline retries and duplicate submissions.
- Test security, backups and deployment.
- Pilot with a limited group before wider rollout.

4. Important Implementation Principles

One shared backend: The mobile app and admin website use the same API; they should not have unrestricted direct access to the production database.

Server-side permissions: Verify every request. A hidden frontend button is not a security boundary.

Offline-first field workflow: Store drafts locally, retry safely and mark a record as synced only after the backend confirms it.

Reliable approvals: Use controlled state transitions and database transactions so conflicting approval decisions cannot both succeed.

Privacy and evidence: Restrict access to location data and media, use private storage, and define retention rules.

Audit history: Record important actions with the actor, timestamp and relevant changes.

5. Development Environment Choices to Confirm

Before generating exact installation commands and the initial runnable code, confirm these four choices:

Question	Available choices
Development computer	Windows 10/11 • Ubuntu/Linux • macOS
PostgreSQL	Local development installation • Managed cloud database
First mobile release	Android first • Android and iOS
Existing prototype	Use as visual reference and rebuild in Flutter • Start fresh

Next deliverable: a configured project skeleton, database schema, authentication flow, setup commands and tests. Features should be clearly labeled as implemented or planned; demo data and placeholder handlers must not be presented as production functionality.